

Smart Contract Security Risks & Crypto Scams

Chapter – 9

Fall 2025

Middle Tennessee State University

Smart Contract Security Risks & Vulnerabilities

1. Default Visibility
2. Front running Attack
3. Third Party Dependency
4. Centralization/Privilege Risk
5. Initial Token Distribution
6. Variables declared as Constant
7. Variables declared as Immutable
8. Integer Overflow and Underflow
9. Reentrancy attack

1. Default Visibility

Introduction:

- 4 main visibility in Solidity (public, internal, external, and private)
- Common mistake developers make is leaving their functions set to “public” by default
- By default, functions are publicly visible
- Visibility is what determines whether a function may or may not be called.

Example:

- Consider a decentralized application (DApp) for managing user profiles. The contract may have a **public** variable storing users' email addresses

Risks Associated:

- Unauthorized access by external contracts
- Developers unintentionally exposing private data within the contract, compromising user privacy.
- Reduced security and privacy

Example of Risks:

- An attacker deploys a rogue contract
- This rogue contract exploits the **public visibility** to access user email addresses
- This leads to privacy breaches and potential misuse of personal information.

1. Default Visibility (Contd...)

Risk: Inadvertently expose sensitive data or functions

Mitigation Techniques:

- **Explicit Visibility:** Always explicitly define visibility (public, private, internal, external) for functions and state variables
- **Minimize Use of Default:** Minimize reliance on default “internal” visibility

Best Practices:

- **Access Control:** Implement access controls (e.g., modifiers in Solidity) to limit interactions with sensitive functions or data.
- **Auditing and Testing:** Thoroughly audit and test contracts to ensure proper visibility is applied where necessary

```
address public owner;  
  
modifier onlyOwner() {  
    require(msg.sender == owner, "Access denied");  
    _;  
}  
  
function sensitiveFunction() internal onlyOwner {  
    // Function logic restricted to only the owner  
}
```



2. Front Running Attack

Introduction:

- Occurs when an attacker exploits the **time delay** between the **initiation of a transaction** and its **inclusion in a block** to gain an unfair advantage

Example:

- Consider a decentralized exchange (DEX) smart contract.
- A user submits a transaction to buy a certain amount of tokens at a specific price.
- Before the transaction is confirmed, an attacker observes it and quickly submits a transaction to buy the same tokens at a slightly higher price.

Risks Associated:

- Attackers can profit at the expense of other users by securing better price

Example of Risks:

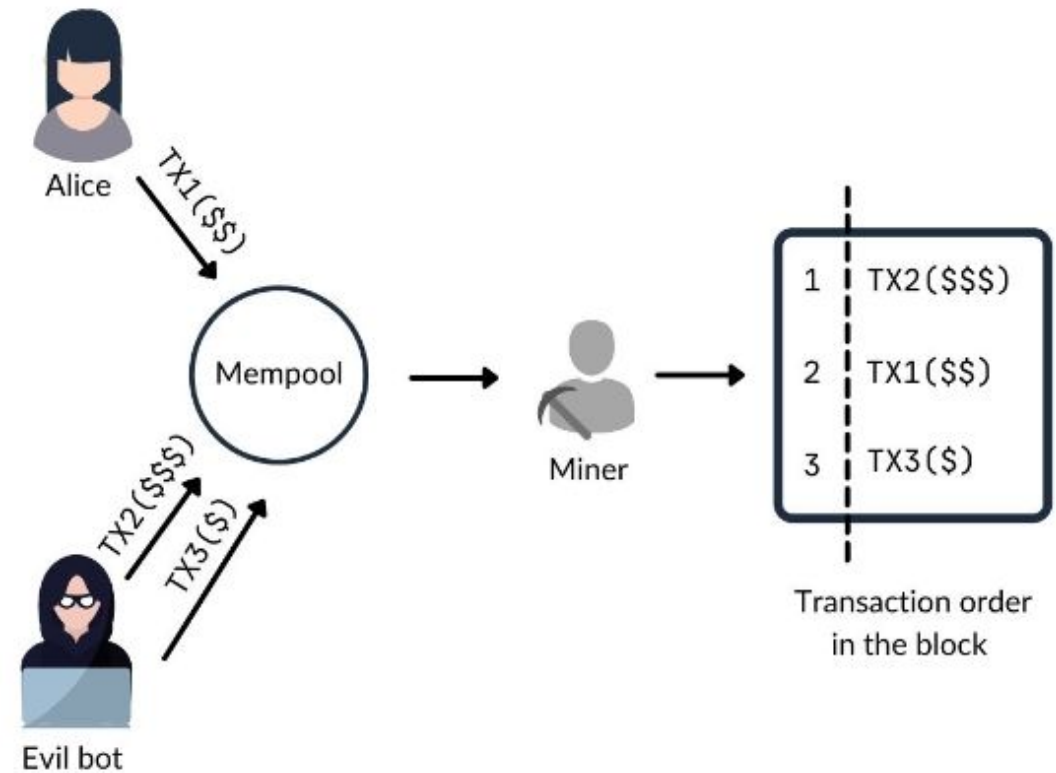
- An attacker observes a user's transaction attempting to purchase tokens at a favorable price.
- The attacker quickly submits a transaction with a slightly higher price, executing the trade before the user's transaction is included in a block.
- This results in the user paying more for the tokens than they had initially intended, while the attacker gains an advantage.

2. Front Running Attack (Contd...)

- Practice of executing a trade or transaction on a blockchain network based on **foreknowledge of a pending transaction**

How does Front Running Occurs?

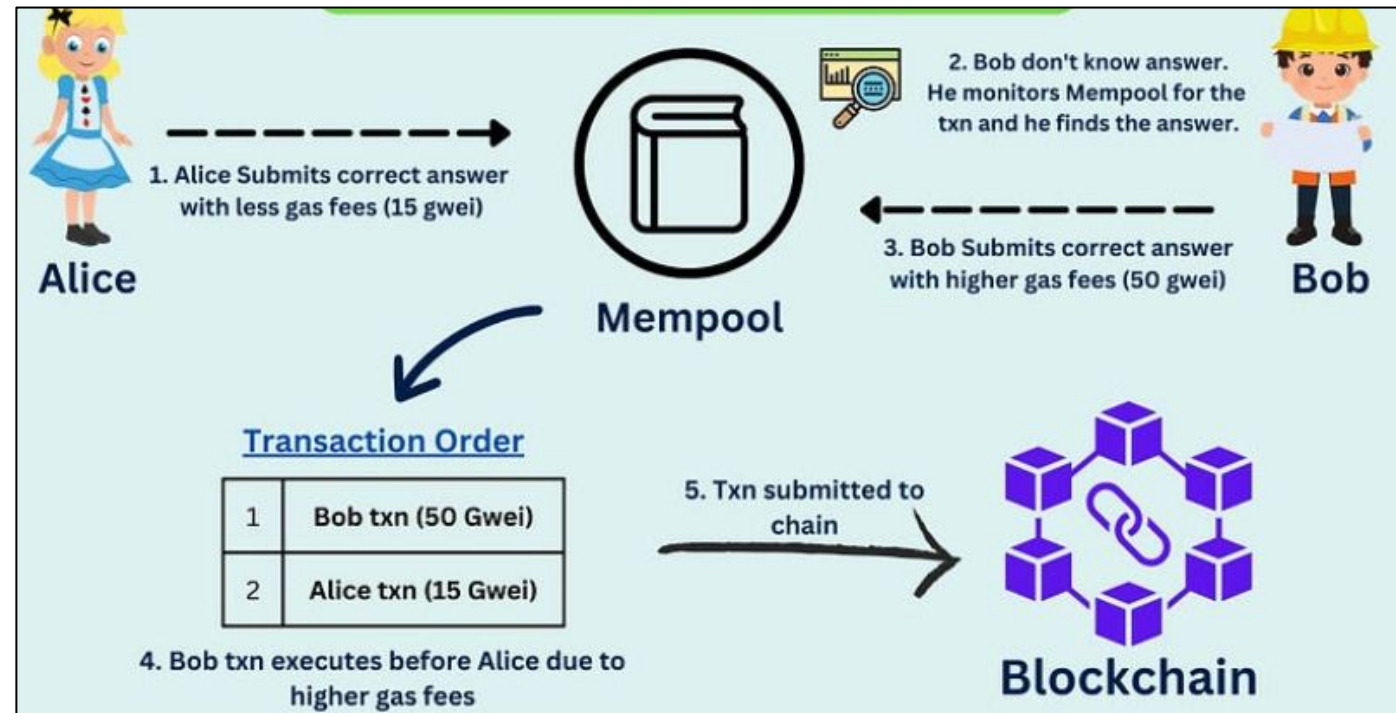
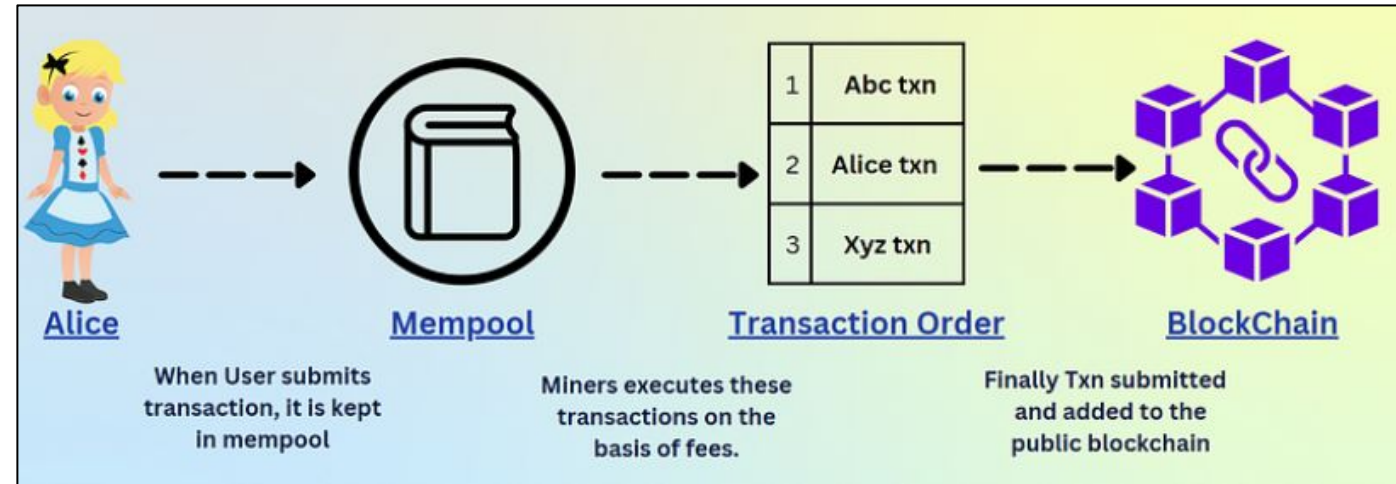
- An attacker monitors pending transactions on the network
- Identifies a transaction that will affect the asset's price
- Submits their own transaction ahead of it to benefit from the price change



2. Front Running Attack (Contd...)

Vulnerabilities that enable Front Running Attack:

- Lack of privacy in blockchain transactions
- Transparency of transaction pool
- Predictable behavior of trading algorithms



2. Front Running Attack (Contd...)

Risk: Exploit transaction delays, impacting fairness and trust

Mitigation Techniques:

- **Order Locking:** Implement time locks operations to prevent front-running opportunities
- **Fee Structures:** Design fee structures to discourage or limit front-running incentives

Best Practices:

- **Encryption and Privacy:** Use encryption or privacy-preserving techniques to obfuscate transaction intentions.
- **Transaction Queuing:** Employ batched or queued transactions to minimize exposure to front-running
- **Off-Peak Periods:** Performing transactions during off-peak periods

3. Third Party Dependency

Introduction:

- Many smart contracts rely on external sources to make informed decisions (Example?)
- Example -> Blockchain Oracle
- This reliance introduces vulnerabilities if the external data source is compromised or unreliable.

Example:

- Consider an insurance smart contract that uses an external weather oracle to determine whether to pay out claims based on weather conditions

Risks Associated:

- Malicious actors may interfere with the data source or oracle, causing it to report inaccurate information.
- Compromised oracles may provide false data, leading to incorrect decisions within the contract.
- If the data source is compromised, it may trigger unwarranted insurance payouts or denials

Example of Risks:

- An attacker manipulates the external weather oracle to falsely report severe weather conditions, resulting in the automatic payout of insurance claims to policyholders who do not meet the actual claim criteria

3. Third Party Dependency (Contd...)

Risk: Smart contracts reliant on external services or oracles are vulnerable to failures or manipulation of these third-party entities

Mitigation Techniques:

- **Oracle Redundancy:** Rely on multiple trusted oracles or data sources to cross-verify and validate external data inputs
- **Data Aggregation:** Implement mechanisms to aggregate data from multiple sources to ensure consistency and reliability

Best Practices:

- **Timeouts and Fail-Safes:** Incorporate timeout mechanisms or fail-safes to handle unresponsive or delayed third-party services.
- **Decentralization of Dependency:** Reduce reliance on a single third-party service.
- **Continuous Monitoring and Auditing:** Regularly monitor third-party services and conduct audits to identify vulnerabilities

4. Centralization/Privilege Risk

Introduction:

- Excessive centralization or privileges within a smart contract
- Leads to single point of failure and potential misuse by privileged individuals

Example:

- Imagine a decentralized voting system that features a single administrator with the authority to manipulate votes or halt elections without requiring consensus from the participants.

Risks Associated:

- Lack of consensus in decision-making
- Manipulation of the contract by privileged individuals or administrators
- Reduced trust in the system's integrity

Example of Risks:

- The single administrator manipulates vote tallies, affecting election outcomes, leading to a lack of trust in the voting system, and potentially causing disputes and dissatisfaction among participants.

5. Initial Token Distribution

Introduction:

- Initial distribution of tokens in a project can significantly impact user trust
- Unequal or unfair token distributions can lead to dissatisfaction among participants.
- Distributed via *constructor* when the smart contract is deployed

Example:

- Consider the launch of a new cryptocurrency project in which the majority of tokens are allocated to a small group of insiders or developers

Risks Associated:

- Resentment among users and early adopters
- Unequal token distribution may discourage active engagement
- Discord and dissatisfaction within the project's community

Example of Risks:

- Early adopters and the wider community become dissatisfied with the project because they perceive the token distribution as unfair and biased in favor of insiders, leading to reduced support and engagement.

4/5. Centralization/Privilege Risks/ Initial Token Distribution (Contd...)

Risk: Centralization or privileged access can compromise decentralization and fairness in smart contracts

Risk: Initial token distribution methods can impact fairness, community perception, and project stability

Mitigation Techniques:

- **Access Control Mechanisms:** Utilize access control mechanisms (e.g., modifiers) to limit privileged functions or data access
- **Multi-Signature Approvals:** Require multi-signature approvals for critical actions, ensuring no single entity has unilateral control.

Best Practices:

- **Token Vesting Schedules:** Apply token vesting schedules to founders or early investors, aligning incentives with the project's long-term success
- **Transparent Processes:** Ensure transparency in decision-making, code development, and fund allocations to build trust within the community.
- **Regular Audits and Reviews:** Conduct regular security audits and community reviews to detect and address centralization risks.

6. Variables declared as Constant

Introduction:

- Using the "constant" keyword inappropriately can result in inflexible contract behavior
- This makes it challenging to adapt to changing circumstances or requirements

Example:

- Imagine a decentralized marketplace with a fixed commission rate declared as constant, making it unadjustable even when market conditions change.

Risks Associated:

- Inability to adapt to changing conditions or market fluctuations
- Reduced flexibility in contract management
- Inflexible contract parameters can lead to inefficiencies and financial losses

Example of Risks:

- As transaction costs rise due to network congestion, the contract's fixed commission rate cannot be adjusted, which can lead to inefficiencies and potentially drive users away.

7. Variables declared as Immutable

Introduction:

- Immutable variables are those whose values cannot be changed after deployment in a smart contract.
- Often used for parameters that should remain constant throughout the contract's lifecycle.

Example:

- Imagine a decentralized voting contract where the total number of votes required for a decision is declared as immutable.

Risks Associated:

- Lack of flexibility: Inability to adjust crucial parameters.
- Potential for outdated parameters in the future.

Example of Risks:

- If the voting quorum is immutable, changes in the community size may render it obsolete, impacting decision-making.

6/7. Variables declared as Constant & Immutable (Contd...)

Risk: Constants and immutable variables may lead to inflexibility and limit adaptability in smart contracts.

Mitigation Techniques:

- **Dynamic Formulas or Logic:** Design contracts with dynamic formulas or logic that can adapt to changing conditions without compromising security
- **Smart Contract Libraries:** Utilize smart contract libraries or external references for certain parameters or data, enabling updates without modifying the core contract

Best Practices:

- **Transparency in Governance Changes:** Ensure transparency and clear communication regarding any changes made to immutable and constant variables
- **Rigorous Testing and Auditing:** Conduct thorough testing and external audits when implementing updates or changes to immutable variables to mitigate potential risks

8. Integer Overflow and Underflow

Introduction:

- Occurs when a smart contract cannot process an operation correctly due to the numbers involved being too large or too small
- Integer Overflow happens when a numeric value **exceeds the maximum representable value** for its data type
- Integer Underflow occurs when a numeric value **goes below the minimum representable** value.

Example:

- Consider a smart contract managing a token's total supply. If an attacker manipulates the supply to overflow or underflow, unexpected consequences may arise.

Risks Associated:

- Incorrect calculations
- Unexpected contract behavior
- Financial losses for users

Example of Risks:

- An attacker exploits an integer overflow, causing the total supply to wrap around and potentially granting themselves an excessive number of tokens.

8. Integer Overflow and Underflow (Contd...)

Risk: Integer overflow/underflow can lead to unexpected behavior, compromising the integrity and security of smart contracts

Mitigation Techniques:

- **Bounds Checking and Constraints:** Apply bounds checking and enforce constraints on input values to ensure they stay within safe ranges.

Best Practices:

- **Safe Arithmetic Libraries:** Utilize safe arithmetic libraries (e.g., SafeMath in Solidity) to perform mathematical operations, preventing overflow/underflow.
- **Constant Monitoring and Testing:** Continuously monitor and extensively test mathematical operations to identify and rectify potential vulnerabilities.

```
pragma solidity ^0.8.0;

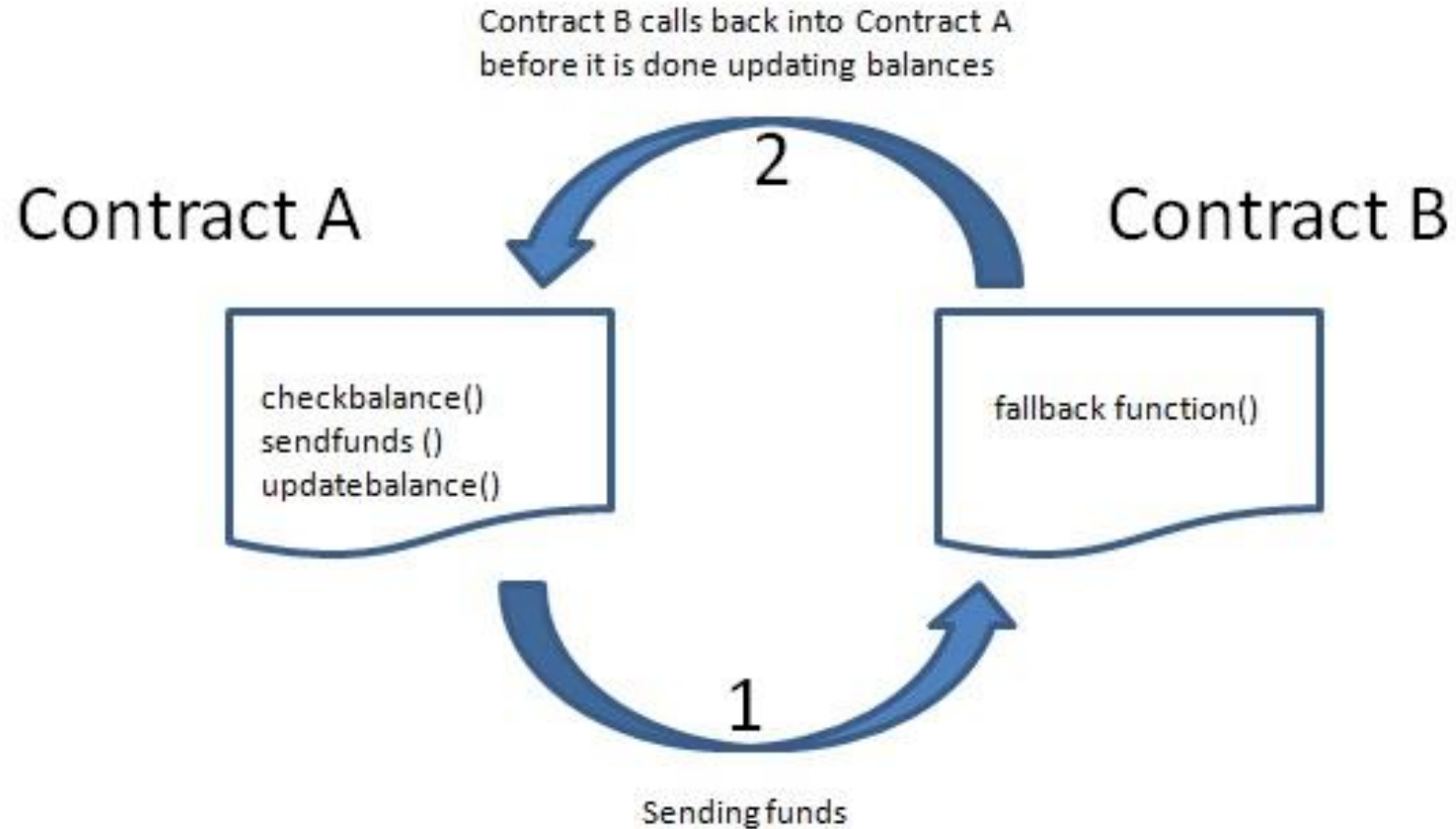
import
"@openzeppelin/contracts/utils/math/SafeMath.sol";

contract MathExample {
    using SafeMath for uint256;

    uint256 public myNumber;

    function safeAdd(uint256 value) public {
        myNumber = myNumber.add(_value);
    }
}
```

9. Reentrancy Attack



9. Reentrancy Attack (Contd...)

Introduction:

- Occurs when a malicious contract exploits the sequential execution of transactions to repeatedly call a vulnerable contract's function
- Suppose a smart contract fails to update its status before sending funds
- Hacker has the potential of repeatedly calling on the withdraw function
- This can cause problems, such as draining funds from a contract
- “DAO hack” in 2016 caused the loss of approx. \$60 million worth of Ethers

Example:

- Consider a crowdfunding contract that releases funds to a project and allows project owners to withdraw. A malicious contract could repeatedly call the withdrawal function before the state is updated.

Risks Associated:

- Unauthorized fund withdrawal.
- Data corruption and unexpected contract state changes
- Financial losses for users.

Example of Risks:

- An attacker exploits a reentrancy vulnerability, draining the crowdfunding contract's funds before the legitimate withdrawal occurs.

9. Reentrancy Attack (Contd...)

Introduction:

- **Risk:** Reentrancy attacks can exploit vulnerable contracts, leading to loss of funds

Mitigation Techniques:

- **Limit External Calls:** Restrict or minimize external calls within critical parts of the contract to reduce exposure to reentrancy vulnerabilities
- **Checks-Effects-Interactions Pattern:** Follow the Checks-Effects-Interactions pattern to ensure that state changes occur before interacting with external contracts

Best Practices:

- **Gas Limits and Fail-Safes:** Apply gas limits and fail-safes in critical functions so that the contract does not run for a longer period of time
- **Reentrancy Guards and Locks:** Use 'reentrancy guards' to prevent multiple recursive calls and limit reentrancy risks

```
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract ReentrancyExample {
    mapping(address => uint256) private balances;

    function withdraw(uint256 _amount) public {
        require(balances[msg.sender] >= _amount, "Insufficient
balance");

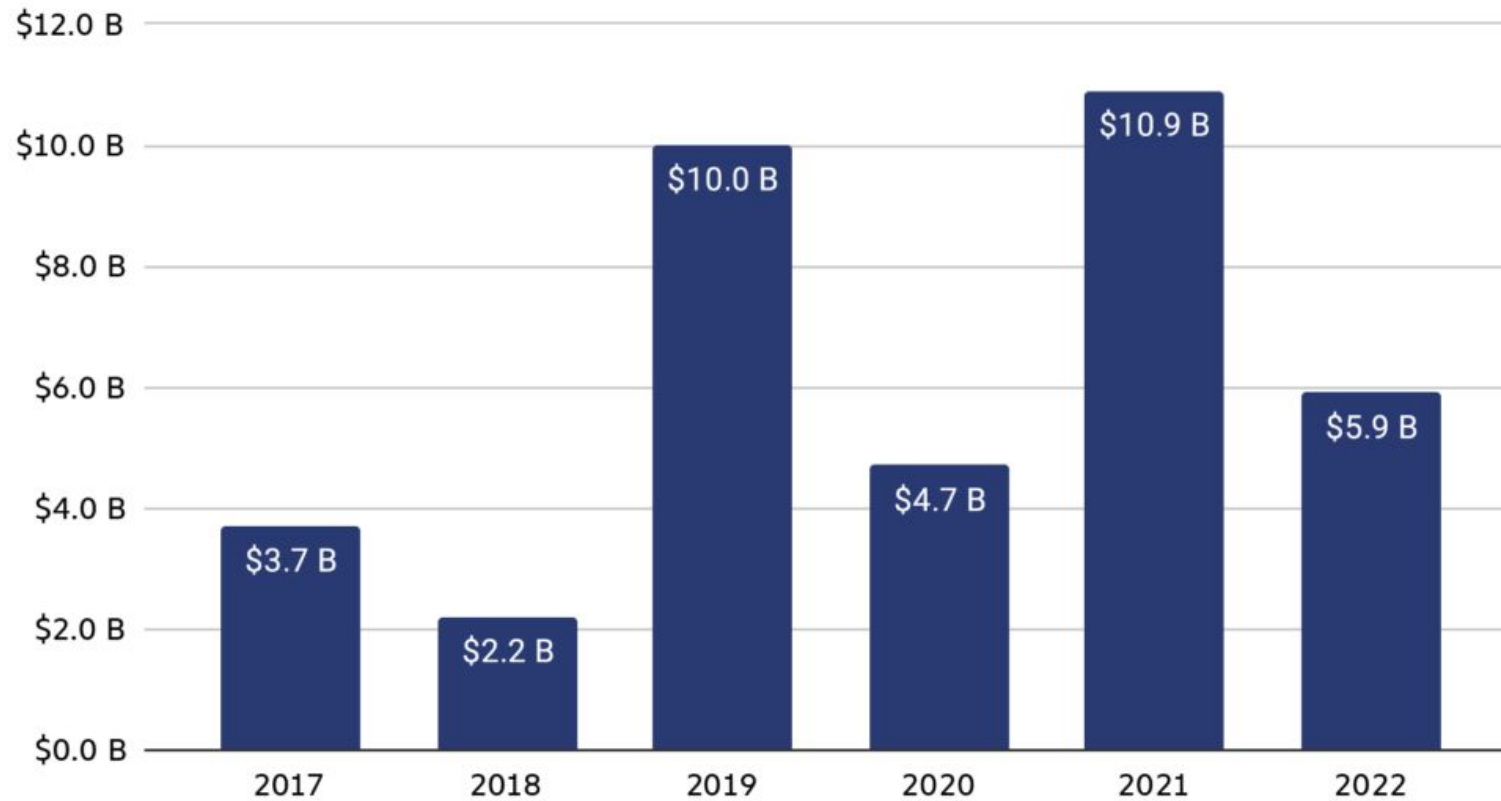
        // CHECKS
        balances[msg.sender] -= _amount; // EFFECTS

        // INTERACTIONS
        (bool success, ) = msg.sender.call{value: _amount}("");
        // External call

        require(success, "Transfer failed");
    }
}
```

Crypto Scams

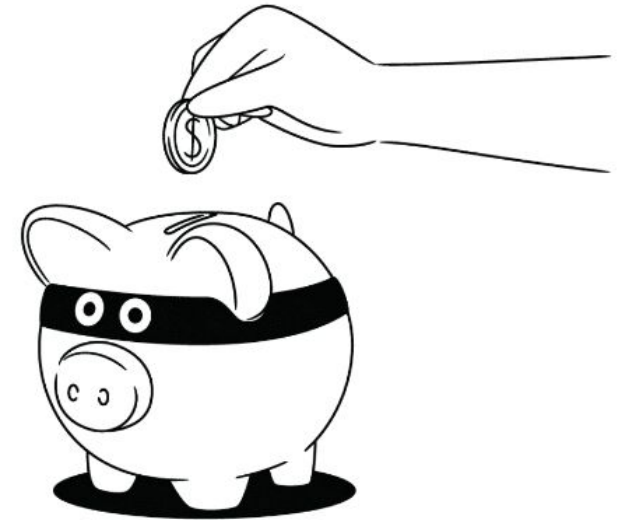
Yearly crypto scam revenue, 2017 - 2022



© Chainalysis

Investment Manager Deception

- Scammers pose as experienced investment managers, boasting massive crypto profits
- Promise investors significant returns on their investments in cryptocurrencies
- Request upfront fees from investors, then abscond with the money without delivering promised returns



Rug Pull Scam

- Pump and Dump
- Scammers “pump up” a new project, nonfungible token (NFT) or coin to get funding
- After the scammers get the money, they “dump” the project and disappear with the collected money
- Smart Contracts prevent customers from selling their cryptocurrency/tokens after purchase
- So, investors are left with a valueless investment
- Famous example: Squid Game Token
 - Price of the token was pumped up
 - Multiple investors could not sell their tokens
 - Price of the token fell from \$2,860 to nil
 - Rugpullers abandoned the project and ran away with \$ 3.3 million



Romance Scam

- As per Federal Trade Commission (FTC), \$185 million have been stolen since 2021
- Con artist sets up fake profiles on dating sites and/or social media sites to entice targets
- “accidentally” DM you on messaging platforms such as WhatsApp and Instagram
- create fake – but convincing-looking – websites
- Pig Butchering
- fattening up the “pig” with small deposits and pretending the victim is netting huge returns



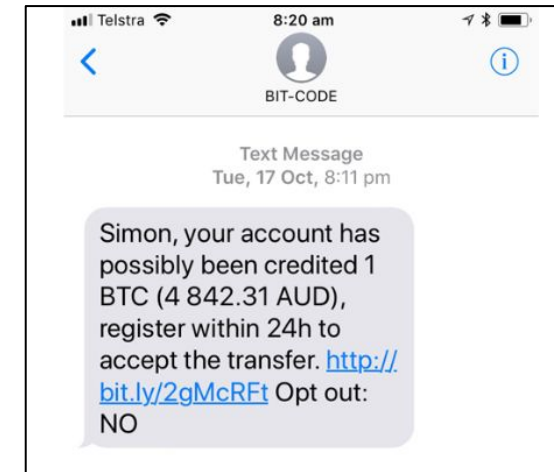
Phishing Scam

- Emails with malicious links to a fake website to gather personal details
- For e.g., cryptocurrency wallet key information – private key
- To avoid phishing scams, never enter secure information from an email link



Social Media Cryptocurrency Giveaway Scam

- Fraudulent posts on social media outlets promising crypto giveaways
- Scammers offer to equal or multiply the cryptocurrency transferred to them
- Include fake celebrity accounts promoting the giveaway to lure people in



Ponzi Schemes

- Pyramid Scheme
- Ponzi schemes pay older investors with the proceeds from new ones
- the “expert” having superior knowledge of a complex and new technology
- the promise of huge profits with little risk
- To get fresh investors, cryptocurrency scammers will lure new investors with cryptocurrency
- \$7.8B lost in crypto Ponzi and pyramid schemes in 2022 – as per trmlabs.com



NFT Scams

- Justin Bieber's Bored Ape NFT was valued at \$1.3 million in 2022; now it's only worth around \$60,000, according to CNBC
- Basketball star Stephen Curry purchased Bored Ape #7990 for 55 ETH (nearly \$180,000 at the time) on Aug 2021. But as of July 2023, the highest bid for the NFT is 30.5 ETH, (\$58,000), according to OpenSea. Nearly 68% drop!
- Music icon Madonna bought Bored Ape #4988 for 180 ETH (around \$466,000 at the time) on March 2022, but the highest bid on the NFT was 28 ETH (a little over \$53,000) as of July 2023, according to OpenSea
- Most NFT fraudsters aim to get access to your NFT account for the private key
- Also, they will create a fake or worthless NFT project and prey upon people's **fear of missing out**



End of Chapter-9